Exp: 7
08/08/26

Flow Control at Data Link Layer using Sliding Window Protocol

Aim:

To write a program to implement flow control at the data link layer using sliding window protocol.

Algorithm:

i) Start the program.

ii) Read the total number of frames to be sent (N), the sender's window size (W), and optionally a frame number.

iii) Initialize base = 0, nextSeq = 0.

iv) Repeat while base < N:

a) Determine the current window as the range [base, base + W - 1].

b) Send all frames with sequence numbers from nextSeq upto (base + W - 1), incrementing nextSeq after each frame.

c. If the lost frames lies within the frames just sent:

1. The receiver discards the lost frame and any subsequent out-of-order frames.

2. The sender times out and retransmits all frames starting from the lost frame.

d. Else :

1. The receiver accepts each frame in order to ~~send~~ sends a cumulative acknowledgement for it.

2. On receiving the ACK for the last frame sent, the sender slides its window forward. base = nextSeq.

v) Repeat step 4 until all N frames have been transmitted and acknowledged.

vi) Stop the program.

Program

```
def simulate (total_frames, window_size, lost_frame) :
    base =0
    next_seq =0
    while base < total_frames :
        window_end = min ( base + window_size - 1, total_frames-1)
        print (50*"-")
        print (f"Sender Window : [Frame {base} - Frame {window_end}
        ]")
        while next_seq < base + window_size and
        next_seq < total_frames :
            print (f"Node A --> Sending Frame {next_seq}")
```

```
        if next_seq == lost_frame:
            print(f"  (Frame {next_seq} Lost in transmit)")
        next_seq +=1
    if lost_frame is not None and base <= lost_frame <
    next_seq:
        print(f" Node B --> Frame {lost_frame} missing.
        Discarding out-of-order frames after it")
        print(f" Node A --> Timeout! Resending window
        starting from Frame {lost_frame}.")
        next_seq = lost_frame
        lost_frame = None
    for f in range(base, next_seq):
        print(f" Node B --> Received frame {f},
        sending ACK {f}")
        print(f" Node A --> ACK {next_seq - 1} received.
        Sliding window forward.")
        base = next_seq
print(50 * "-")
print(f"All {total_frames} frames successfully transmitted")
```

Exp: 8
13/8/26

```
=== SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N) ===

Enter total number of frames to send     : 8
Enter sender's window size               : 4
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss)           : 5

-----------------------------------------
Sender Window : [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node B --> Received Frame 3, sending ACK 3
Node A --> ACK 3 received. Sliding window forward.

-----------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
        (Frame 5 LOST in transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out-of-order frames after it.
Node A --> Timeout! Resending window starting from Frame 5.

-----------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 5
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 4, sending ACK 4
Node B --> Received Frame 5, sending ACK 5
Node B --> Received Frame 6, sending ACK 6
Node B --> Received Frame 7, sending ACK 7
Node A --> ACK 7 received. Sliding window forward.

-----------------------------------------
All 8 frames successfully transmitted from Node A to Node B.
```

Result :

The program to implement flow control at data link layer using sliding window protocol has been completed.